

DSA Sheet 3 - Problems 41 to 60

--- Problem 41: Binary Search ---

Problem Statement

Given an array of integers `nums` which is sorted in ascending order, and an integer `target`, write a function to search for `target` in `nums`.

If the `target` exists, return its index. Otherwise, return `-1`.

You must write an algorithm with $O(\log n)$ runtime complexity.

Input Format

First line contains the sorted integer array `nums`.

Second line contains the integer `target`.

Output Format

Print the index of the target element if found.

Otherwise, print `-1`.

Constraints

$1 \leq \text{nums.length} \leq 10^4$

$-10^4 \leq \text{nums}[i], \text{target} \leq 10^4$

All integers in `nums` are unique.

`nums` is sorted in ascending order.

Example 1

Input

`nums = [-1, 0, 3, 5, 9, 12]`

`target = 9`

Output

4

Explanation

9 is present in the array at index 4.

Example 2

Input

`nums = [-1, 0, 3, 5, 9, 12]`

`target = 2`

Output

-1

Explanation

2 is not present in the array, so return `-1`.

--- Problem 42: Search Insert Position ---

Problem Statement

Given a sorted array of distinct integers and a target value, return the index if the target is found.

If not found, return the index where it would be inserted in order.

You must write an algorithm with $O(\log n)$ runtime complexity.

Input Format

First line contains the sorted integer array `nums`.

Second line contains the integer `target`.

Output Format

Print the index of the target if it exists.

Otherwise, print the index where it should be inserted.

Constraints

$1 \leq \text{nums.length} \leq 10^4$

$-10^4 \leq \text{nums}[i] \leq 10^4$

`nums` contains distinct values sorted in ascending order.

$-10^4 \leq \text{target} \leq 10^4$

DSA Sheet 3 - Problems 41 to 60

Example 1

Input

nums = [1, 3, 5, 6]

target = 5

Output

2

Explanation

5 is already present in the array at index 2.

Example 2

Input

nums = [1, 3, 5, 6]

target = 2

Output

1

Explanation

2 is not present in the array. It should be inserted at index 1.

Example 3

Input

nums = [1, 3, 5, 6]

target = 7

Output

4

Explanation

7 is greater than all elements, so it should be inserted at index 4.

--- Problem 43: Sqrt(x) ---

Problem Statement

Given a non-negative integer x , return the square root of x rounded down to the nearest integer.

The returned integer should be non-negative as well.

You must not use any built-in exponent function or operator.

Input Format

A single integer x .

Output Format

Print the square root of x rounded down to the nearest integer.

Constraints

$0 \leq x \leq 2^{31} - 1$

Example 1

Input

$x = 4$

Output

2

Explanation

The square root of 4 is 2, so we return 2.

Example 2

Input

$x = 8$

Output

2

Explanation

The square root of 8 is 2.828....

DSA Sheet 3 - Problems 41 to 60

Since we return only the integer part, the answer is 2.

--- Problem 44: Search in Rotated Sorted Array ---

Problem Statement

There is an integer array `nums` sorted in ascending order (with distinct values).

Before being passed to your function, `nums` is possibly rotated at an unknown pivot index `k` such that the resulting array becomes:

`[nums[k], nums[k+1], ..., nums[n-1], nums[0], nums[1], ..., nums[k-1]]`

Given the array `nums` after rotation and an integer `target`, return the index of `target` if it exists in the array. Otherwise, return `-1`.

You must write an algorithm with $O(\log n)$ runtime complexity.

Input Format

First line contains the rotated sorted array `nums`.

Second line contains the integer `target`.

Output Format

Print the index of the target element if found.

Otherwise, print `-1`.

Constraints

$1 \leq \text{nums.length} \leq 5000$

$-10^4 \leq \text{nums}[i] \leq 10^4$

All values of `nums` are unique.

`nums` is an ascending array that is possibly rotated.

$-10^4 \leq \text{target} \leq 10^4$

Example 1

Input

`nums = [4, 5, 6, 7, 0, 1, 2]`

`target = 0`

Output

4

Explanation

0 is present in the array at index 4.

Example 2

Input

`nums = [4, 5, 6, 7, 0, 1, 2]`

`target = 3`

Output

-1

Explanation

3 is not present in the array, so return -1.

Example 3

Input

`nums = [1]`

`target = 0`

Output

-1

Explanation

The array contains only one element and the target does not exist.

--- Problem 45: Find First and Last Position of Element ---

Problem Statement

Given an array of integers `nums` sorted in non-decreasing order, find the starting and ending position of a given target value.

If the target is not found in the array, return `[-1, -1]`.

You must write an algorithm with $O(\log n)$ runtime complexity.

Input Format

DSA Sheet 3 - Problems 41 to 60

First line contains the sorted integer array nums.

Second line contains the integer target.

Output Format

Print the starting and ending index of the target in the form [start, end].

If the target is not present, print [-1, -1].

Constraints

$0 \leq \text{nums.length} \leq 10^5$

$-10^9 \leq \text{nums}[i] \leq 10^9$

nums is sorted in non-decreasing order.

$-10^9 \leq \text{target} \leq 10^9$

Example 1

Input

nums = [5, 7, 7, 8, 8, 10]

target = 8

Output

[3, 4]

Explanation

The first occurrence of 8 is at index 3, and the last occurrence is at index 4.

Example 2

Input

nums = [5, 7, 7, 8, 8, 10]

target = 6

Output

[-1, -1]

Explanation

6 is not present in the array, so return [-1, -1].

Example 3

Input

nums = []

target = 0

Output

[-1, -1]

Explanation

The array is empty, so the target cannot be found.

--- Problem 46: Find Minimum in Rotated Sorted Array ---

Problem Statement

Suppose an array of length n sorted in ascending order is rotated between 1 and n times.

For example, the array nums = [0,1,2,4,5,6,7] might become:

[4,5,6,7,0,1,2] if rotated 4 times.

[0,1,2,4,5,6,7] if rotated 7 times.

Notice that rotating an array [a[0], a[1], a[2], ..., a[n-1]] one time results in the array:

[a[n-1], a[0], a[1], ..., a[n-2]].

Given the sorted rotated array nums of unique elements, return the minimum element of this array.

You must write an algorithm that runs in $O(\log n)$ time.

Input Format

A single line containing the rotated sorted array nums.

Output Format

Print the minimum element present in the array.

Constraints

DSA Sheet 3 - Problems 41 to 60

$n == \text{nums.length}$
 $1 \leq n \leq 5000$
 $-5000 \leq \text{nums}[i] \leq 5000$
All integers in nums are unique.
nums is sorted and rotated between 1 and n times.

Example 1

Input
nums = [3,4,5,1,2]

Output

1
Explanation
The original array was [1,2,3,4,5] and it was rotated 3 times, so the minimum element is 1.

Example 2

Input
nums = [4,5,6,7,0,1,2]

Output

0
Explanation
The array was rotated 4 times and the minimum element in the array is 0.

Example 3

Input
nums = [11,13,15,17]

Output

11
Explanation
The array is already sorted and not effectively rotated, so the minimum element is 11.

--- Problem 47: Find Peak Element (1D) ---

Problem Statement

A peak element is an element that is strictly greater than its neighbors.

Given a 0-indexed integer array nums, find a peak element and return its index.

If the array contains multiple peaks, return the index of any one of the peaks.

You may imagine that $\text{nums}[-1] = \text{nums}[n] = -\infty$. In other words, an element is always considered greater than a neighbor that is outside the array.

You must write an algorithm that runs in $O(\log n)$ time.

Input Format

A single line containing the integer array nums.

Output Format

Print the index of any peak element.

Constraints

$1 \leq \text{nums.length} \leq 1000$
 $-2^{31} \leq \text{nums}[i] \leq 2^{31} - 1$
 $\text{nums}[i] \neq \text{nums}[i + 1]$ for all valid i

Example 1

Input
nums = [1, 2, 3, 1]

Output

2
Explanation
In the given array, the element 3 is greater than both its neighbors 2 and 1, so it is a peak element. Its index is 2.

DSA Sheet 3 - Problems 41 to 60

Example 2

Input

nums = [1, 2, 1, 3, 5, 6, 4]

Output

5

Explanation

The element 6 is greater than its neighbors 5 and 4, so it is a peak element. Its index is 5.

Another valid answer could be index 1 because element 2 is also a peak.

--- Problem 48: Find a Peak Element II (2D) ---

Problem Statement

A peak element in a 2D grid is an element that is strictly greater than all of its adjacent neighbors to the left, right, top, and bottom.

Given a 0-indexed $m \times n$ matrix `mat` where no two adjacent cells are equal, find any peak element and return its position $[i, j]$.

You may assume that the entire matrix is surrounded by an outer perimeter with the value -1 in each cell.

You must write an algorithm that runs in $O(m \log n)$ or $O(n \log m)$ time.

Input Format

A 2D integer matrix `mat`.

Output Format

Print the position of any peak element in the form `[row, column]`.

Constraints

$m == \text{mat.length}$

$n == \text{mat}[i].\text{length}$

$1 \leq m, n \leq 500$

$1 \leq \text{mat}[i][j] \leq 10^5$

No two adjacent cells are equal.

Example 1

Input

`mat = [[1,4],[3,2]]`

Output

`[0,1]`

Explanation

In the given matrix, the element 4 at position `[0,1]` is greater than all of its adjacent neighbors.

The element 3 at position `[1,0]` is also a peak element, so `[1,0]` would also be a valid answer.

Example 2

Input

`mat = [[10,20,15],[21,30,14],[7,16,32]]`

Output

`[1,1]`

Explanation

The element 30 at position `[1,1]` is greater than its adjacent neighbors 20, 21, 14, and 16, so it is a peak element.

--- Problem 49: Capacity To Ship Packages Within D Days ---

Problem Statement

A conveyor belt has packages that must be shipped from one port to another within `days` days.

The i -th package on the conveyor belt has a weight of `weights[i]`.

Each day, we load the ship with packages in the order given by `weights`. We may not load more weight than the maximum weight capacity of the ship.

Return the least weight capacity of the ship that will result in all the packages being shipped within `days` days.

Input Format

First line contains the integer array `weights`.

Second line contains the integer `days`.

Output Format

DSA Sheet 3 - Problems 41 to 60

Print the minimum ship capacity required to ship all packages within the given number of days.

Constraints

$1 \leq \text{days} \leq \text{weights.length} \leq 5 * 10^4$

$1 \leq \text{weights}[i] \leq 500$

Example 1

Input

weights = [1,2,3,4,5,6,7,8,9,10]

days = 5

Output

15

Explanation

A ship capacity of 15 is the minimum required to ship all packages within 5 days while maintaining the given order.

Example 2

Input

weights = [3,2,2,4,1,4]

days = 3

Output

6

Explanation

With a ship capacity of 6, all packages can be shipped in exactly 3 days while maintaining the given order.

Example 3

Input

weights = [1,2,3,1,1]

days = 4

Output

3

Explanation

The minimum ship capacity required to ship all packages within 4 days is 3.

--- Problem 50: Painter's Partition Problem ---

Problem Statement

Given 2 integers A and B, and an array of integers C of size N.

A represents the number of painters.

B represents the time taken by one painter to paint 1 unit of board.

C[i] represents the length of the i-th board.

You have to paint all N boards [C0, C1, C2, ..., CN-1].

There are A painters available, and each painter takes B units of time to paint 1 unit of board.

Calculate and return the minimum time required to paint all boards under the following constraints:

A painter will only paint contiguous sections of boards.

A board cannot be split between painters.

Return the answer modulo 10000003.

Input Format

First line contains integer A - number of painters.

Second line contains integer B - time taken to paint one unit of board.

Third line contains array C representing lengths of boards.

Output Format

Print the minimum time required to paint all boards modulo 10000003.

Constraints

$1 \leq A \leq 1000$

$1 \leq B \leq 10^6$

$1 \leq N \leq 10^5$

DSA Sheet 3 - Problems 41 to 60

$1 \leq C[i] \leq 10^6$

Example 1

Input

A = 2

B = 5

C = [1, 10]

Output

50

Explanation

If one painter paints both boards, the time required is: $(1 + 10) \times 5 = 55$

If painter 1 paints board 1 and painter 2 paints board 10, then: $\max(1 \times 5, 10 \times 5) = 50$

So, the minimum possible time is 50.

Example 2

Input

A = 10

B = 1

C = [1, 8, 11, 3]

Output

11

Explanation

Since there are enough painters, boards can be distributed optimally.

The maximum time taken by any painter is 11, so the minimum total time required is 11.

--- Problem 51: Minimize Max Distance to Gas Station ---

Problem Statement

We have a horizontal number line.

On the number line, there are N gas stations placed at positions:

stations[0], stations[1], ..., stations[N-1]

Now, we add k more gas stations so that the maximum distance between adjacent gas stations is minimized.

We have to find the smallest possible value of this maximum distance.

Return the answer accurate to 6 decimal places.

Note: stations is given in strictly increasing order.

Input Format

First line contains the array stations.

Second line contains the integer k.

Output Format

Print the minimum possible maximum distance between adjacent gas stations up to 6 decimal places.

Constraints

$1 \leq \text{stations.size()} \leq 10^5$

$0 \leq \text{stations}[i] \leq 10^8$

$0 \leq k \leq 10^6$

Example 1

Input

stations = [1, 2, 3, 4, 5]

k = 2

Output

1.000000

Explanation

The distance between every adjacent gas station is already 1.

Adding extra gas stations does not reduce the maximum distance further, so the minimum possible maximum distance remains 1.000000.

DSA Sheet 3 - Problems 41 to 60

Example 2

Input

stations = [3, 6, 12, 19, 33]

k = 3

Output

6.000000

Explanation

By placing 3 additional gas stations optimally, the maximum distance between adjacent gas stations can be minimized to 6.000000.

--- Problem 52: Allocate Books (Minimum Largest Pages) ---

Problem Statement

Given an array `arr[]` of integer numbers, where `arr[i]` represents the number of pages in the *i*-th book.

There are *m* students, and the task is to allocate all the books to the students under the following conditions:

Each book contains some pages.

A student can be allocated only contiguous books.

Each book must be allocated to exactly one student.

The allocation should minimize the maximum number of pages assigned to a student.

If the allocation is not possible, return -1.

Input Format

First line contains integer *n* - number of books.

Second line contains integer *m* - number of students.

Third line contains array `arr` representing pages in each book.

Output Format

Print the minimum possible value of the maximum pages assigned to a student.

If allocation is not possible, print -1.

Constraints

$2 \leq n \leq 10^3$

$1 \leq m \leq 10^3$

$1 \leq arr[i] \leq 10^4$

Sum of all `arr[i]` does not exceed 10^9

Where: *n* denotes the number of books; *m* denotes the number of students.

Example 1

Input

n = 4

m = 2

arr = [12, 34, 67, 90]

Output

113

Explanation

Possible allocations are:

[12, 34] and [67, 90] -> maximum pages = 157

[12, 34, 67] and [90] -> maximum pages = 113

The minimum possible maximum is 113.

Example 2

Input

n = 3

m = 4

arr = [10, 20, 30]

Output

-1

Explanation

There are more students than books, so allocation is not possible.

--- Problem 53: Aggressive Cows ---

DSA Sheet 3 - Problems 41 to 60

Problem Statement

You are given an array stalls[] with unique integers representing the positions of stalls.

You are also given an integer k representing the number of aggressive cows.

The task is to assign stalls to the cows such that the minimum distance between any two cows is maximized.

Return the largest possible minimum distance.

Input Format

First line contains the array stalls.

Second line contains the integer k.

Output Format

Print the maximum possible minimum distance between any two cows.

Constraints

$2 \leq \text{stalls.size()} \leq 10^6$

$0 \leq \text{stalls}[i] \leq 10^8$

$2 \leq k \leq \text{stalls.size()}$

Example 1

Input

stalls = [1, 2, 4, 8, 9]

k = 3

Output

3

Explanation

Place the first cow at stall 1, the second at stall 4, the third at stall 8.

The minimum distance between any two cows is 3, which is the largest possible.

Example 2

Input

stalls = [10, 1, 2, 7, 5]

k = 3

Output

4

Explanation

After sorting the stalls: [1, 2, 5, 7, 10]

One optimal placement is: Cow 1 -> 1, Cow 2 -> 5, Cow 3 -> 10

The minimum distance between any two cows is 4, which is the maximum possible.

--- Problem 54: Binary Tree Preorder Traversal ---

Problem Statement

Given the root of a binary tree, return the preorder traversal of its node values.

In preorder traversal, nodes are visited in the following order:

Root Node

Left Subtree

Right Subtree

Input Format

A binary tree represented as root = [values].

null represents the absence of a node.

Output Format

Print the preorder traversal of the binary tree as an array.

Constraints

The number of nodes in the tree is in the range [0, 100]

$-100 \leq \text{Node.val} \leq 100$

DSA Sheet 3 - Problems 41 to 60

Example 1

Input

root = [1, null, 2, 3]

Output

[1, 2, 3]

Explanation

Visit root node 1, move to the right child 2, visit the left child of 2, which is 3.

So, the preorder traversal is [1, 2, 3].

Example 2

Input

root = [1, 2, 3, 4, 5, null, 8, null, null, 6, 7, 9]

Output

[1, 2, 4, 5, 6, 7, 3, 8, 9]

Explanation

Preorder traversal: Visit Root, traverse Left Subtree completely, traverse Right Subtree completely.

Thus, the traversal order is [1, 2, 4, 5, 6, 7, 3, 8, 9].

--- Problem 55: Maximum Depth of Binary Tree ---

Problem Statement

Given the root of a binary tree, your task is to find the maximum depth (height) of the tree.

Note:

The maximum depth or height of the tree is the number of edges in the longest path from the root node to the deepest leaf node.

Input Format

A binary tree represented as root = [values].

N or null represents the absence of a node.

Output Format

Print the height (maximum depth) of the binary tree.

Constraints

$1 \leq \text{number of nodes} \leq 3 * 10^4$

$0 \leq \text{node} \rightarrow \text{data} \leq 10^5$

Example 1

Input

root = [12, 8, 18, 5, 11]

Tree Structure:

```
    12
   / \
  8   18
 / \
5   11
```

Output

2

Explanation

One of the longest paths is: 12 -> 8 -> 5

This path contains 2 edges, so the maximum depth of the tree is 2.

Example 2

Input

root = [1, 2, 3, 4, N, 10, 5, N, N, N, N, 6, 7]

Tree Structure:

```
    1
   / \
  2   3
 / \
10  5
```

DSA Sheet 3 - Problems 41 to 60

4 10 5

/ \

6 7

Output

3

Explanation

One of the longest paths is: 1 -> 3 -> 5 -> 6

This path contains 3 edges, so the maximum depth of the tree is 3.

--- Problem 56: Same Tree (Identical Binary Trees) ---

Problem Statement

Given two binary trees with their root nodes r1 and r2, return true if both trees are identical, otherwise return false.

Two trees are considered identical when:

They have the same structure.

Corresponding nodes contain the same data.

Input Format

First line contains the binary tree r1.

Second line contains the binary tree r2.

N or null represents the absence of a node.

Output Format

Print true if both binary trees are identical.

Otherwise, print false.

Constraints

1 <= number of nodes <= 10⁵

1 <= node->data <= 10⁹

Example 1

Input

r1 = [1, 2, 3, 4]

r2 = [1, 2, 3, 4]

Output

true

Explanation

Both binary trees have the same structure and the same values at each corresponding node.

Hence, the trees are identical.

Example 2

Input

r1 = [1, 2, 3, 4]

r2 = [1, 2, 3, N, N, 4]

Output

false

Explanation

The structures of the two trees are different, so the trees are not identical.

--- Problem 57: Symmetric Tree ---

Problem Statement

Given the root of a binary tree, check whether it is a mirror of itself (i.e., symmetric around its center).

A binary tree is symmetric if:

The left subtree is a mirror reflection of the right subtree.

Both structure and node values match symmetrically.

Input Format

A binary tree represented as root = [values].

null represents the absence of a node.

DSA Sheet 3 - Problems 41 to 60

Output Format

Print true if the tree is symmetric.
Otherwise, print false.

Constraints

The number of nodes in the tree is in the range [1, 1000]
 $-100 \leq \text{Node.val} \leq 100$

Example 1

Input

root = [1, 2, 2, 3, 4, 4, 3]

Tree Structure:

```
    1
   /\
  2 2
 /\ /\
3 4 4 3
```

Output

true

Explanation

The left subtree is the mirror image of the right subtree.
Both the structure and node values match symmetrically, so the tree is symmetric.

Example 2

Input

root = [1, 2, 2, null, 3, null, 3]

Tree Structure:

```
    1
   /\
  2 2
   \ \
    3 3
```

Output

false

Explanation

The left and right subtrees are not mirror images of each other, so the tree is not symmetric.

--- Problem 58: Diameter of Binary Tree ---

Problem Statement

Given the root of a binary tree, return the length of the diameter of the tree.
The diameter of a binary tree is the length of the longest path between any two nodes in the tree.
This path may or may not pass through the root.
The length of a path between two nodes is represented by the number of edges between them.

Input Format

A binary tree represented as root = [values].
null represents the absence of a node.

Output Format

Print the diameter of the binary tree.

Constraints

The number of nodes in the tree is in the range [1, 10^4]
 $-100 \leq \text{Node.val} \leq 100$

Example 1

Input

root = [1, 2, 3, 4, 5]

DSA Sheet 3 - Problems 41 to 60

Tree Structure:

```
  1
 / \
2   3
 / \
4   5
```

Output

3

Explanation

The longest path in the tree can be: 4 -> 2 -> 1 -> 3 or 5 -> 2 -> 1 -> 3

Both paths contain 3 edges, so the diameter of the tree is 3.

Example 2

Input

root = [1, 2]

Tree Structure:

```
  1
 /
 2
```

Output

1

Explanation

The longest path is 2 -> 1, which contains 1 edge. Hence, the diameter is 1.

--- Problem 59: Boundary Traversal of Binary Tree ---

Problem Statement

Given the root of a Binary Tree, return its boundary traversal in the following order:

Left Boundary: Nodes from the root to the leftmost non-leaf node, preferring the left child over the right child.

Leaf Nodes: All leaf nodes from left to right covering every leaf in the tree.

Reverse Right Boundary: Nodes from the root to the rightmost non-leaf node, preferring the right child over the left child, excluding leaves and added in reverse order.

Note:

The root is included once.

Leaf nodes are added separately to avoid repetition.

The right boundary is traversed in reverse order.

Input Format

A binary tree represented as root = [values].

N or null represents the absence of a node.

Output Format

Print the boundary traversal of the binary tree as an array.

Constraints

1 <= number of nodes <= 10⁵

1 <= node->data <= 10⁵

Example 1

Input

root = [1, 2, 3, 4, 5, 6, 7, N, N, 8, 9, N, N, N, N]

Tree Structure:

```
  1
 / \
 2   3
 / \ / \
4  5 6  7
 / \

```

DSA Sheet 3 - Problems 41 to 60

8 9

Output

[1, 2, 4, 8, 9, 6, 7, 3]

Explanation

Left boundary traversal: 1 -> 2 -> 4

Leaf nodes: 8, 9, 6, 7

Reverse right boundary: 3

So, the final boundary traversal is [1, 2, 4, 8, 9, 6, 7, 3].

Example 2

Input

root = [1, N, 2, N, 3, N, 4, N, N]

Tree Structure:

```
1
 \
 2
  \
   3
    \
     4
```

Output

[1, 4, 3, 2]

Explanation

Left boundary: 1

Leaf node: 4

Reverse right boundary: 3, 2

Hence, the boundary traversal is [1, 4, 3, 2].

--- Problem 60: Lowest Common Ancestor of a Binary Tree ---

Problem Statement

Given a binary tree, find the Lowest Common Ancestor (LCA) of two given nodes in the tree.

The Lowest Common Ancestor of two nodes p and q is defined as the lowest node in the tree that has both p and q as descendants (where a node can be a descendant of itself).

Input Format

First line contains the binary tree represented as root = [values].

Second line contains integer p.

Third line contains integer q.

null represents the absence of a node.

Output Format

Print the value of the Lowest Common Ancestor (LCA) of nodes p and q.

Constraints

The number of nodes in the tree is in the range [2, 10⁵]

-10⁹ ≤ Node.val ≤ 10⁹

All node values are unique.

p ≠ q

Both p and q exist in the tree.

Example 1

Input

root = [3,5,1,6,2,0,8,null,null,7,4]

p = 5

q = 1

Tree Structure:

```
  3
 / \
```

DSA Sheet 3 - Problems 41 to 60

```
5  1
/\  /\
6  2 0 8
  /\
  7  4
```

Output

3

Explanation

Node 3 is the lowest node that has both nodes 5 and 1 as descendants.

Hence, the Lowest Common Ancestor is 3.

Example 2

Input

root = [3,5,1,6,2,0,8,null,null,7,4]

p = 5

q = 4

Tree Structure:

```
3
/\ 
5  1
/\  /\
6  2 0 8
  /\
  7  4
```

Output

5

Explanation

Node 5 is an ancestor of node 4, and a node can be a descendant of itself.

Therefore, the Lowest Common Ancestor is 5.